```
(*
                            CS 51 Lab 19
                  Greenberg-Hastings Cellular Automaton
 https://en.wikipedia.org/wiki/Greenberg Hastings_cellular_automaton

Implementation of the Greenberg-Hastings model described in the
article

    Greenberg, J. M., and S. P. Hastings. "Spatial Patterns for
    Discrete Models of Diffusion in Excitable Media." SIAM Journal on
    Applied Mathematics 34, no. 3 (1978): 515 23.
    http://www.jstor.org/stable/2100950.
 *)

(*
    ************************************************************
    WARNING: If you have photosensitive epilepsy, you should avoid
    viewing the Greenberg-Hastings cellular automaton. It exhibits
    rapidly flashing visual patterns.
    ************************************************************
 *)

module G = Graphics ;;

(* Automaton parameters *)
let cGRID_SIZE = 100 ;;       (* width and height of grid in cells *)
let cSPARSITY = 25 ;;         (* inverse of proportion of cells initially live *)

(* Rendering parameters *)
let cCOLOR_LEGEND = G.rgb 173 106 108 ;;  (* color for textual legend *)
let cSIDE = 8 ;;              (* width and height of cells in pixels *)
let cRENDER_FREQUENCY = 1     (* how frequently grid is rendered (in ticks) *) ;;
let cFONT = Some "-adobe-times-bold-r-normal--34-240-100-100-p-177-iso8859-9"

type gh_state =
  | Resting
  | Excited
  | Refractory

(* offset index off -- Returns the `index` offset by `off` allowing
   for wraparound. *)
let rec offset (index : int) (off : int) : int =
  if index + off < 0 then
    cGRID_SIZE - (offset ~-index ~-off)
```

```
45    else
46      (index + off) mod cGRID_SIZE ;;
47
48  (* gh_update grid i j -- *)
49  let gh_update (grid : gh_state array array) (i : int) (j : int) =
50    let neighbors = ref 0 in
51    for i' = ~-1 to ~+1 do
52      for j' = ~-1 to ~+1 do
53        (* only consider the four neighbors sharing an index *)
54        if (i' = 0 || j' = 0) && not (i' = 0 && j' = 0) then
55          let i_ind = offset i i' in
56          let j_ind = offset j j' in
57          neighbors := !neighbors
58                       + match grid.(i_ind).(j_ind) with
59                         | Resting -> 0
60                         | Refractory -> 0
61                         | Excited -> 1
62      done
63    done;
64    match grid.(i).(j) with
65    | Excited -> Refractory
66    | Refractory -> Resting
67    | Resting -> if !neighbors > 0 then Excited else Resting ;;
68
69  (* gh_random_state () -- Returns a random cell state. *)
70  let gh_random_state () =
71    let states = [Excited; Refractory; Resting; Resting; Resting; Resting] in
72    List.nth states (Random.int (List.length states))
73
74  module GHSpec : (Cellular.AUT_SPEC
75                   with type state = gh_state) =
76    struct
77      type state = gh_state
78      let name : string = "Greenberg-Hastings"
79      let initial : state = Resting
80      let grid_size : int = cGRID_SIZE
81      let random_state = gh_random_state
82      let update = gh_update
83      let cell_color (cell : state) : G.color =
84        match cell with
85        | Resting -> G.rgb 245 240 246
86        | Excited -> G.rgb 56 95 113
87        | Refractory -> G.rgb 43 65 98
88      let side_size : int = cSIDE
89      let legend_color : G.color = cCOLOR_LEGEND
90      let font : string option = cFONT
```

```
    let render_frequency : int = cRENDER_FREQUENCY
  end ;;

module Aut = Cellular.Automaton (GHSpec) ;;

(*.....................................................................
  Running the automaton and displaying the results
 *)

(* main seed -- Runs the automaton using the provided random `seed`
   (for replicability), displaying updates to the graphics window. *)
let main seed =

  Random.init seed;      (* initialize randomness *)
  Aut.graphics_init ();  (* ... and graphics window *)

  (* Initialize the grid to a simple half-line of excited and
  refractory cells, as per the discussion in th article cited at the
  top of the file. This generates the typical spiral artifacts
  described in the paper. *)
  let initial_grid = Aut.fresh_grid () in
  for i = cGRID_SIZE / 2 to cGRID_SIZE - 1 do
    initial_grid.(i).(cGRID_SIZE / 2) <- Refractory;
    initial_grid.(i).(cGRID_SIZE / 2 + 1) <- Excited
  done;

  Aut.run_grid initial_grid ;;

(* Run the automaton with user-supplied seed *)
let _ =
  if Array.length Sys.argv <= 1 then
    Printf.printf "Usage: %s <seed>\n  where <seed> is integer seed\n"
                  Sys.argv.(0)
  else
    main (int_of_string Sys.argv.(1)) ;;

```